Homework 11

Haoran Xu

2026-11-08

Question 1: Implementing K-means

I use seed **432** throughout. The 10 observations have two independent $N(0,1)$ coordinates, and the initial assignment is drawn uniformly at random from $\{1,2\}$, redrawn until both clusters are nonempty.

```
set.seed(432)
X <- matrix(rnorm(20), nrow = 10, ncol = 2)
colnames(X) <- c("x1", "x2")
random_init <- function(n, K) {                # nonempty random assignment
  repeat {
    C <- sample(K, n, replace = TRUE)
    if (length(unique(C)) == K) return(C)
  }
}
C0 <- random_init(nrow(X), K = 2)
cbind(round(X, 3), init = C0)
#>            x1     x2 init
#>  [1,] -0.658 -0.035    1
#>  [2,]  1.400  0.282    1
#>  [3,]  0.601 -0.698    1
#>  [4,]  0.633 -0.917    1
#>  [5,]  0.474  1.145    1
#>  [6,]  0.392 -1.586    1
#>  [7,] -0.672  0.512    2
#>  [8,] -0.483 -0.569    2
#>  [9,] -0.080 -0.793    1
#> [10,] -0.189 -0.410    2
```

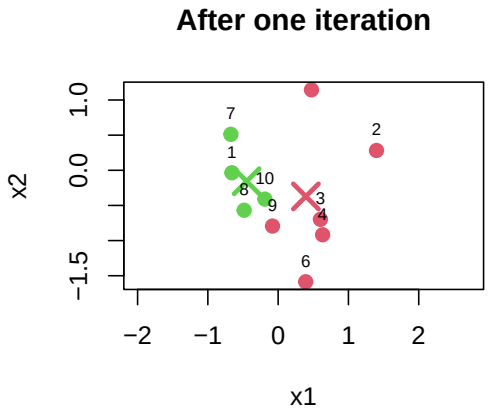
(a) One iteration

The two steps are (1) compute each cluster mean m_k from the current assignment, and (2) reassign every x_i to the nearest mean. I compute all squared Euclidean distances at once with the identity $\|x_i - m_k\|^2 = \|x_i\|^2 + \|m_k\|^2 - 2\, x_i^\top m_k$.

```
cluster_means <- function(X, C, K, old = NULL) {
  M <- matrix(NA_real_, K, ncol(X), dimnames = list(paste0("m", 1:K), colnames(X)))
  for (k in 1:K) {
    if (any(C == k)) M[k, ] <- colMeans(X[C == k, , drop = FALSE])
    else M[k, ] <- old[k, ]                    # empty cluster: keep previous mean
  }
  M
}
sq_dist <- function(X, M)                      # n x K matrix of squared distances
  outer(rowSums(X^2), rowSums(M^2), "+") - 2 * X %*% t(M)
```

```r
M1 <- cluster_means(X, C0, K = 2)            # step 1: means of the initial clusters
C1 <- apply(sq_dist(X, M1), 1, which.min)    # step 2: nearest-mean assignment
M1
#>         x1      x2
#> m1  0.3946 -0.3718
#> m2 -0.4481 -0.1557
rbind(initial = C0, updated = C1)
#>         [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9] [,10]
#> initial    1    1    1    1    1    1    2    2    1     2
#> updated    2    1    1    1    1    1    2    2    1     2
```

```r
plot_clusters <- function(X, C, M, main = "") {
  plot(X, col = C + 1, pch = 19, cex = 1.2, main = main, asp = 1)
  points(M, col = seq_len(nrow(M)) + 1, pch = 4, cex = 2.2, lwd = 3)
  text(X, labels = seq_len(nrow(X)), pos = 3, cex = 0.7)
}
plot_clusters(X, C1, M1, "After one iteration")
```



After one iteration 1 observation (number 1) switched clusters. The crosses are the means of the *initial* clusters, which were used for the reassignment; they are updated at the start of the next iteration.

(b) A general K-means function

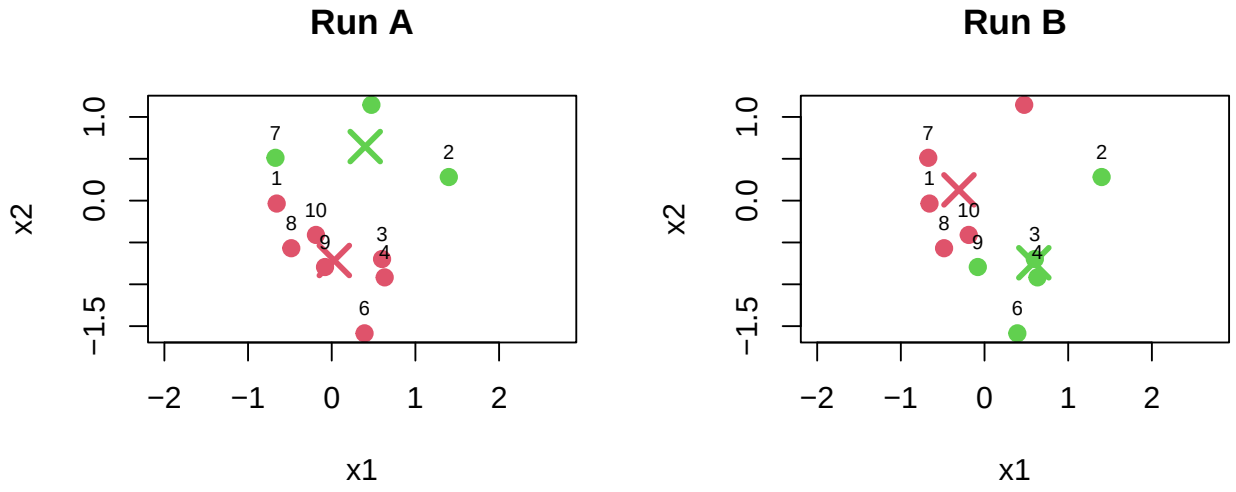
The function below takes any data matrix, a number of clusters K, an initial assignment (a nonempty random one by default), and a maximum number of iterations. In each iteration it recomputes the means (an empty cluster keeps its previous mean) and reassigns the points, stopping as soon as the assignment does not change. Before returning it recomputes the means from the final assignment and evaluates the within-cluster sum of squares $\sum_k \sum_{i:C(i)=k} \|x_i - m_k\|^2$.

```r
my_kmeans <- function(X, K, init = NULL, max.iter = 100) {
  X <- as.matrix(X)
  C <- if (is.null(init)) random_init(nrow(X), K) else init
  M <- cluster_means(X, C, K)
  for (iter in seq_len(max.iter)) {
    M <- cluster_means(X, C, K, old = M)            # step 1: update means
    C_new <- apply(sq_dist(X, M), 1, which.min)     # step 2: reassign
    if (identical(C_new, C)) break
    C <- C_new
  }
  M <- cluster_means(X, C, K, old = M)              # means of the final assignment
```

```
  wss <- sum(sq_dist(X, M)[cbind(seq_len(nrow(X)), C)])
  list(cluster = C, centers = M, wss = wss, iter = iter,
       converged = iter < max.iter || identical(C_new, C))
}
```

(c) Two random starts

```
set.seed(1);  initA <- random_init(10, 2)
set.seed(2);  initB <- random_init(10, 2)
rbind(initA, initB)
#>       [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9] [,10]
#> initA    1    2    1    1    2    1    1    1    2     2
#> initB    1    1    2    2    2    2    1    1    1     2
fitA <- my_kmeans(X, K = 2, init = initA)
fitB <- my_kmeans(X, K = 2, init = initB)
plot_clusters(X, fitA$cluster, fitA$centers, "Run A")
plot_clusters(X, fitB$cluster, fitB$centers, "Run B")
```




```
rbind(runA = fitA$cluster, runB = fitB$cluster)
#>      [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9] [,10]
#> runA    1    2    1    1    2    1    2    1    1     1
#> runB    1    2    2    2    1    2    1    1    2     1
fitA$centers; fitB$centers
#>          x1      x2
#> m1 0.03095 -0.7155
#> m2 0.40039  0.6462
#>          x1      x2
#> m1 -0.3056  0.1286
#> m2  0.5891 -0.7426
c(wss_A = fitA$wss, wss_B = fitB$wss, iter_A = fitA$iter, iter_B = fitB$iter)
#>  wss_A  wss_B iter_A iter_B
#>  5.555  5.837  2.000  2.000
```

The two runs end in **different** partitions with **different** objective values: run A separates observations 2 and 5 (upper right) from the rest, while run B splits the cloud roughly along a diagonal, and run A's within-cluster sum of squares (5.5546) is smaller than run B's (5.8366). Both runs converged in two iterations, so neither is a failure

of the algorithm. K-means is a coordinate-descent method: each of the two steps weakly decreases the objective (the cluster mean minimizes the within-cluster squared distances, and the nearest-mean reassignment can only lower them further), and there are finitely many partitions, so it always converges, but only to a *local* minimum, a partition that no single mean-update/reassignment can improve. Which local minimum we reach depends on the start. With only 10 points there is no dominant cluster structure, so the objective has several shallow basins and different random starts fall into different ones. Repeating my function from 100 random starts makes this concrete:

```
set.seed(432)
wss100 <- replicate(100, round(my_kmeans(X, K = 2)$wss, 4))
table(wss100)
#> wss100
#> 5.5515 5.5546 5.6483 5.8366 5.8744  6.024 6.1459 6.3676 6.5954 6.5963
#>      4     13     20     17      1      9      3      1     17     11
#> 6.7655
#>      4
```

There are 11 distinct local minima; run A's solution is the second-best and run B's is reached about as often. The usual remedy is exactly this: run K-means from many random starts and keep the partition with the smallest within-cluster sum of squares (here 5.5515), which is what `nstart` does in standard implementations.

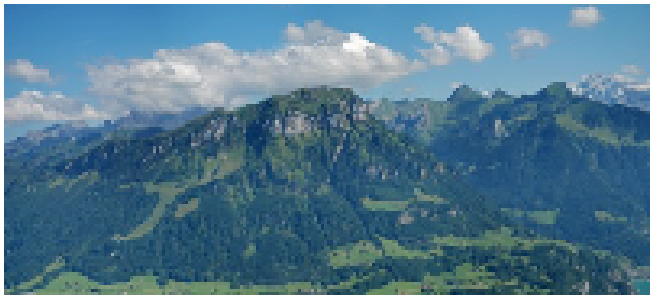
Question 2: Image color compression

(a) The image and its pixel matrix

The image is a panorama of the Swiss Alps from Fronalpstock (Wikimedia Commons, `Fronalpstock_big.jpg`), which I like for its mix of sky, cloud, forest, meadow and rock. I reduced the 10109×4542 original to 480 pixels wide before storing it in `data/alps.png`, and below I resize it in R to at most 200×200 pixels by nearest-neighbour subsampling (taking equally spaced rows and columns), which keeps the aspect ratio.

```
library(png)
img <- readPNG("data/alps.png")[, , 1:3]          # drop alpha channel if present
dim(img)                                           # height x width x RGB, values in [0, 1]
#> [1] 215 480   3
resize_nn <- function(img, max.side = 200) {
  s <- max.side / max(dim(img)[1:2])
  r <- round(seq(1, dim(img)[1], length.out = round(dim(img)[1] * s)))
  c <- round(seq(1, dim(img)[2], length.out = round(dim(img)[2] * s)))
  img[r, c, ]
}
small <- resize_nn(img)
dim(small)
#> [1]  90 200   3
show_img <- function(img, main = "") {
  plot(c(0, ncol(img)), c(0, nrow(img)), type = "n", axes = FALSE,
       xlab = "", ylab = "", asp = 1, main = main, cex.main = 0.9)
  rasterImage(img, 0, 0, ncol(img), nrow(img), interpolate = FALSE)
}
par(mar = c(0, 0, 1.5, 0))
show_img(small, sprintf("Resized image: %d x %d pixels", nrow(small), ncol(small)))
```

Resized image: 90 x 200 pixels



```
pix <- cbind(R = as.vector(small[, , 1]), G = as.vector(small[, , 2]),
             B = as.vector(small[, , 3]))
dim(pix)
#> [1] 18000     3
```

The resized image has 90 rows and 200 columns, so the data matrix has 18000 observations (one per pixel) and 3 variables, the red, green and blue intensities on the $[0, 1]$ scale.

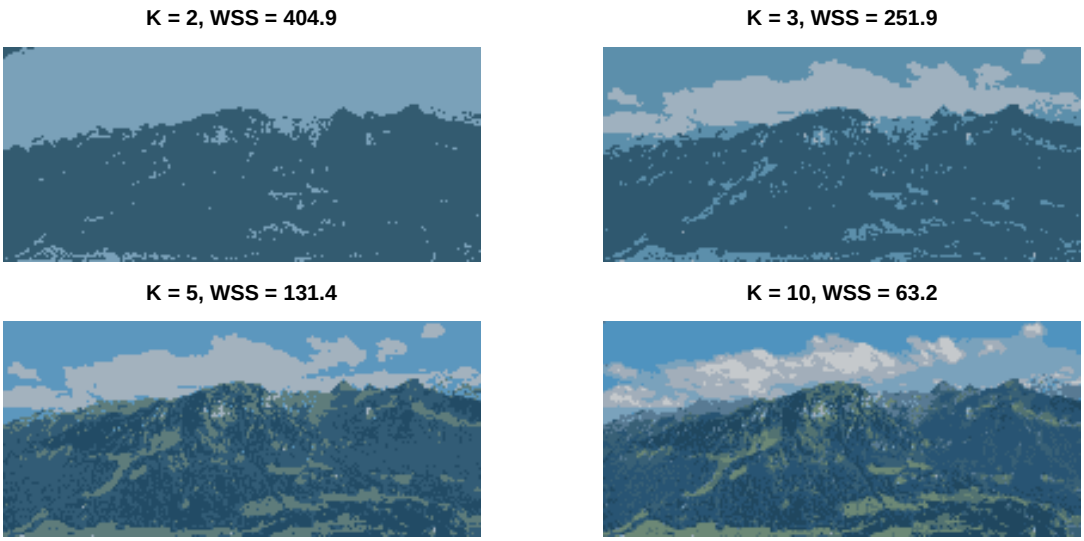
(b) K-means with $K = 2, 3, 5, 10$

Each fit starts from a random nonempty assignment with seed 432 and runs my function from Question 1. Replacing every pixel by its cluster mean gives the compressed image; all images are drawn on the same $[0, 1]$ RGB scale.

```
Ks <- c(2, 3, 5, 10)
fits <- lapply(Ks, function(K) { set.seed(432); my_kmeans(pix, K, max.iter = 200) })
names(fits) <- paste0("K", Ks)
for (K in names(fits)) {
  cat(sprintf("\n%s:  WSS = %.2f  (%d iterations)\n", K, fits[[K]]$wss, fits[[K]]$iter))
  print(round(fits[[K]]$centers, 3))
}
#>
#> K2:  WSS = 404.86  (10 iterations)
#>        R     G     B
#> m1 0.199 0.355 0.441
#> m2 0.479 0.631 0.726
#>
#> K3:  WSS = 251.92  (33 iterations)
#>        R     G     B
#> m1 0.361 0.562 0.670
#> m2 0.624 0.695 0.748
#> m3 0.185 0.343 0.435
#>
#> K5:  WSS = 131.45  (25 iterations)
#>        R     G     B
#> m1 0.367 0.488 0.483
#> m2 0.197 0.360 0.462
#> m3 0.133 0.293 0.396
#> m4 0.640 0.700 0.746
#> m5 0.360 0.591 0.744
#>
#> K10:  WSS = 63.18  (117 iterations)
#>         R     G     B
#> m1  0.345 0.479 0.577
#> m2  0.158 0.329 0.451
```

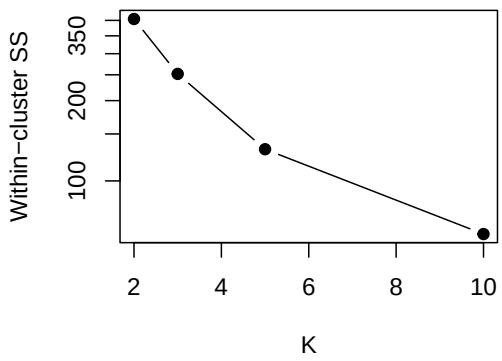
```
#> m3  0.773 0.787 0.801
#> m4  0.321 0.445 0.443
#> m5  0.312 0.575 0.750
#> m6  0.635 0.683 0.728
#> m7  0.478 0.636 0.738
#> m8  0.221 0.378 0.463
#> m9  0.424 0.534 0.461
#> m10 0.125 0.280 0.373
```

```
reconstruct <- function(fit, dims) {
  out <- fit$centers[fit$cluster, ]                # each pixel -> its cluster mean
  out <- pmin(pmax(out, 0), 1)
  array(out, dim = c(dims[1], dims[2], 3))
}
par(mfrow = c(2, 2), mar = c(0, 0, 1.5, 0))
for (K in names(fits))
  show_img(reconstruct(fits[[K]], dim(small)),
           sprintf("K = %s, WSS = %.1f", sub("K", "", K), fits[[K]]$wss))
```



(c) Choosing K

```
wss <- sapply(fits, `[[`, "wss")
par(mar = c(4, 4, 1, 1))
plot(Ks, wss, type = "b", pch = 19, xlab = "K", ylab = "Within-cluster SS", log = "y")
```



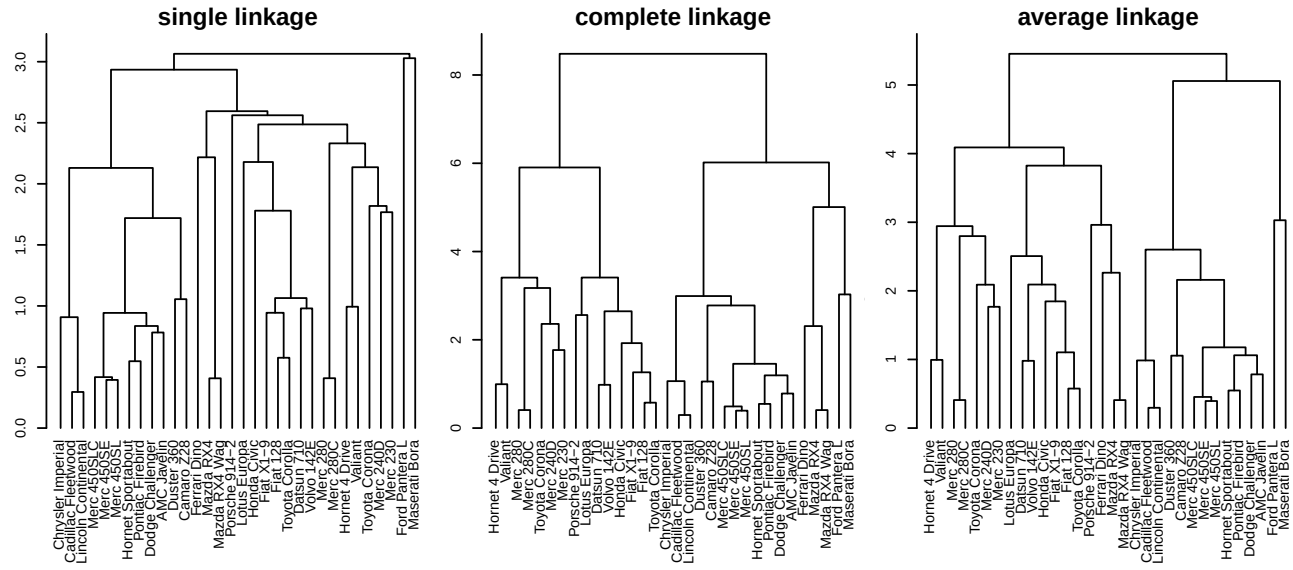
```
round(rbind(WSS = wss, `relative to K = 2` = wss / wss[1]), 3)
#>                          K2      K3      K5    K10
#> WSS                 404.9 251.924 131.447 63.181
#> relative to K = 2   1.0   0.622   0.325  0.156
```

The within-cluster sum of squares must decrease with K, so I look at where the extra clusters stop buying much. The drop from $K = 2$ to $K = 3$ and to $K = 5$ is large, while going from 5 to 10 colors reduces the objective by a smaller amount per added cluster. The images tell the same story: with $K = 2$ the picture is essentially sky versus mountain, with $K = 3$ the clouds, the blue sky and the dark slopes separate, and with $K = 5$ the sunlit meadows, shaded forest, rock faces, sky and clouds are all visible, which makes the scene recognisable. $K = 10$ is noticeably smoother (the haze on the far ridges and the gradations of green come back) but each extra color now mostly refines shading rather than adding a new feature. For a compression task I would choose $K = 5$: it captures the main colour structure of the scene at a storage cost of 5 colors plus about 2.3 bits per pixel, whereas $K = 10$ doubles the palette for a modest further gain. If visual fidelity matters more than compression, $K = 10$ is the better of the four.

Question 3: Hierarchical clustering of cars

(a) Three linkages on standardized variables

```
cars <- read.csv("data/mtcars.csv")
Xc <- scale(cars[, -1]); rownames(Xc) <- cars$car
round(colMeans(Xc), 3); round(apply(Xc, 2, sd), 3)
#>  mpg  cyl disp   hp drat   wt qsec   vs   am gear carb
#>    0    0    0    0    0    0    0    0    0    0    0
#>  mpg  cyl disp   hp drat   wt qsec   vs   am gear carb
#>    1    1    1    1    1    1    1    1    1    1    1
D <- dist(Xc)                                    # Euclidean distance on standardized data
hc <- list(single = hclust(D, "single"), complete = hclust(D, "complete"),
           average = hclust(D, "average"))
par(mfrow = c(1, 3), mar = c(0, 3, 2, 0), cex = 0.5)
for (m in names(hc))
  plot(hc[[m]], main = paste(m, "linkage"), xlab = "", sub = "", hang = -1, cex.main = 1.6)
```



All three methods agree on the tightest pairs (the two Mercedes 450 variants, the Fiat 128 / Toyota Corolla / Honda Civic group, the big American V8s) because those are decided by the smallest distances, but they organise

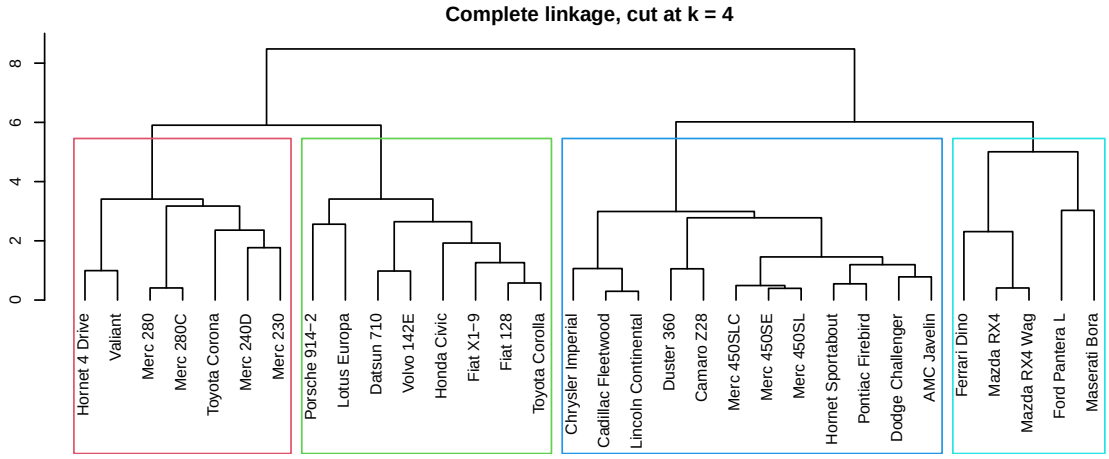
the higher levels very differently.

- **Single linkage** merges clusters by their closest pair, so it *chains*: most cars join one growing cluster one at a time, and the dendrogram is a long staircase whose only clear outliers are the Maserati Bora and Ford Pantera L (high horsepower sports cars) and the Lincoln/Cadillac/Chrysler trio of very heavy cars. Cutting it gives one giant cluster plus a few singletons, which is not a useful partition.
- **Complete linkage** merges by the farthest pair, which favours compact, balanced clusters. Its top split (height 8.5, far above the next merge at 6.0) separates the 17 cars with large engines from the 15 ordinary cars; the next two merges split the five high-performance sports cars (Maserati, Pantera, Ferrari, the two Mazda RX4s) from the twelve V8s, and the ordinary cars into light manual cars and heavier automatic sedans.
- **Average linkage** sits between the two: its first split also isolates the twelve V8s, and it then peels off the Pantera/Maserati pair, but the Mazdas and the Ferrari are absorbed into the ordinary-car branch. Its merge heights decrease more gradually than with complete linkage, so the number of clusters is less obvious.

(b) A chosen partition

I use **complete linkage with four clusters**. The top of its dendrogram has merges at heights 8.5, 6.0, 5.9 and then 5.0: cutting between 5.9 and 5.0 (the next clear gap after the top split) gives four groups, whereas three clusters would separate two merges that happen at essentially the same height. The four groups are also the ones a car buyer would recognise. Average linkage with four clusters gives the same partition except that the two Mazda RX4s and the Ferrari Dino join the light-car group.

```
grp <- cutree(hc$complete, k = 4)
par(mar = c(0, 3, 2, 0), cex = 0.6)
plot(hc$complete, main = "Complete linkage, cut at k = 4", xlab = "", sub = "", hang = -1)
rect.hclust(hc$complete, k = 4, border = 2:5)
```




```
split(names(grp), grp)
#> $`1`
#> [1] "Mazda RX4"      "Mazda RX4 Wag"  "Ford Pantera L" "Ferrari Dino"
#> [5] "Maserati Bora"
#>
#> $`2`
#> [1] "Datsun 710"     "Fiat 128"       "Honda Civic"    "Toyota Corolla"
#> [5] "Fiat X1-9"      "Porsche 914-2"  "Lotus Europa"   "Volvo 142E"
#>
#> $`3`
#> [1] "Hornet 4 Drive" "Valiant"        "Merc 240D"      "Merc 230"
#> [5] "Merc 280"       "Merc 280C"      "Toyota Corona"
#>
```

```
#> $`4`
#>  [1] "Hornet Sportabout"   "Duster 360"          "Merc 450SE"
#>  [4] "Merc 450SL"          "Merc 450SLC"         "Cadillac Fleetwood"
#>  [7] "Lincoln Continental" "Chrysler Imperial"   "Dodge Challenger"
#> [10] "AMC Javelin"         "Camaro Z28"          "Pontiac Firebird"
means <- aggregate(cars[, -1], by = list(cluster = grp), FUN = mean)
means$n <- as.vector(table(grp))
knitr::kable(means, digits = 2, caption = "Cluster means on the original variable scales.")
```

Table 1: Cluster means on the original variable scales.

cluster	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb	n
1	18.50	6.80	223.40	198.80	3.84	3.00	15.62	0.00	1	4.60	5.20	5
2	28.08	4.00	93.61	81.88	4.18	2.04	18.45	0.88	1	4.25	1.50	8
3	20.74	5.14	175.11	102.14	3.57	3.19	19.97	1.00	0	3.57	2.14	7
4	15.05	8.00	357.62	194.17	3.12	4.10	17.14	0.00	0	3.00	3.08	12

The groups have a clear interpretation.

- **Cluster 1 (5 cars)**, the high-performance sports cars: Maserati Bora, Ford Pantera L, Ferrari Dino and the two Mazda RX4s. They have by far the most horsepower per unit weight, the fastest quarter mile (`qsec` 15.6 s), 4 or 5 gears, 5 carburetors on average and all manual transmissions, with middling fuel economy.
- **Cluster 2 (8 cars)**, the light economy and small sports cars: Fiat 128, Honda Civic, Toyota Corolla, Datsun 710, Fiat X1-9, Porsche 914-2, Lotus Europa, Volvo 142E. All 4-cylinder and all manual, the smallest engines (94 cubic inches, 82 hp), the lightest weight (about 2,000 lb) and the best fuel economy (28 mpg).
- **Cluster 3 (7 cars)**, the conventional family sedans: Hornet 4 Drive, Valiant, Toyota Corona and the Mercedes 230/240D/280/280C. Mixed 4- and 6-cylinder straight engines (`vs` = 1 for all), all automatic, moderate size and power, and the slowest quarter mile (20 s), with fuel economy in between (21 mpg).
- **Cluster 4 (12 cars)**, the big American V8s: Cadillac, Lincoln, Chrysler Imperial, the Mercedes 450s and the muscle cars. All 8 cylinders, the largest displacement (358 cubic inches) and weight (4,100 lb), all automatic with 3 gears and the worst fuel economy (15 mpg).

The table shows the trade-offs behind the split: engine size and weight increase from cluster 2 to 3 to 4 while `mpg` falls, and cluster 1 stands apart not by size but by power, gearing and speed. The sports cars and V8s have similar horsepower, which is why they share a branch until height 6, but they differ in weight, gears, transmission and `qsec`.

Why standardization matters. Euclidean distance adds squared differences across variables, so a variable's influence is proportional to its variance. On the raw scale `disp` ranges from 71 to 472 cubic inches and `hp` from 52 to 335, whereas `vs`, `am` are 0/1 and `drat`, `wt` vary by only a few units. Without standardization the distance would be almost entirely determined by displacement and horsepower, and the dendrogram would just sort cars by engine size, throwing away the information in fuel economy, weight, transmission and gearing. Standardizing gives every variable the same variance so each contributes comparably to the clustering (the unit of measurement, cubic inches versus litres, no longer changes the answer).